Visione Artificiale

Raffaella Lanzarotti

Where are we?

First part:
Image formation and Early vision

- Image formation
  - Geometric Camera Models
  - Color spaces
- Image Processing
  - Punctual and spatial processing
  - Feature Extraction
- Reconstruction
  - Camera calibration
  - Stereo Vision
  - Structure from Motion and RGB-d Cameras
  - Optical flow and Tracking

Second part:
Machine learning for CV

- Linear Neural Network
- Multi Layer Perceptron
- Convolutional Neural Networks
- Recurrent Neural Networks
- Transformers
- Variational Auto-Encoders
- Generative Adversarial Networks
- **Graph Neural Networks**
- Self-supervised learning
- Vision Language Models

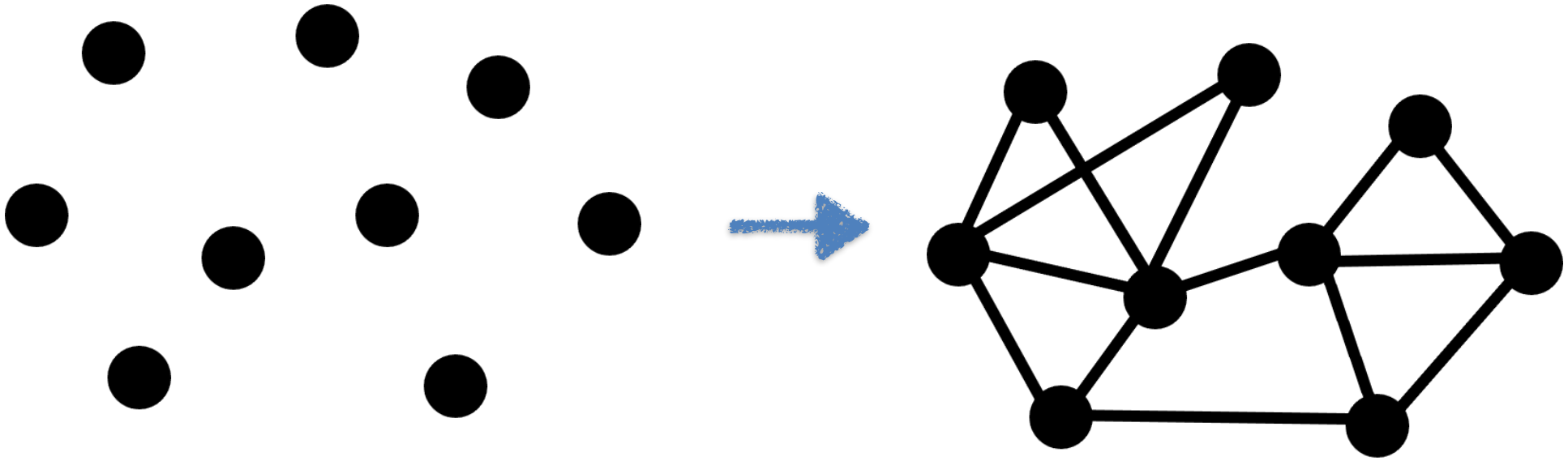
Credits, Yure Leskovec

Why Graphs?

Chapter 13, Bishop

Graphs

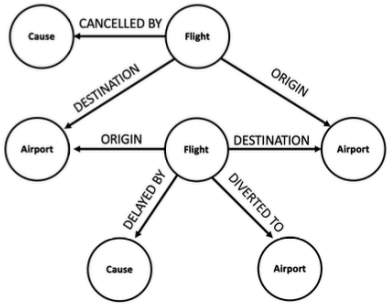
Graphs are a general language for describing and analyzing **structured data with relations/interactions**



Isolated Entities

Graph: relations between entities

Many applications - Natural graphs



Event Graphs

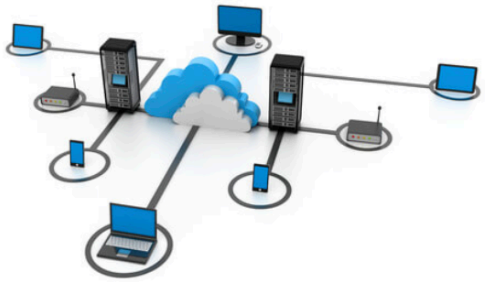
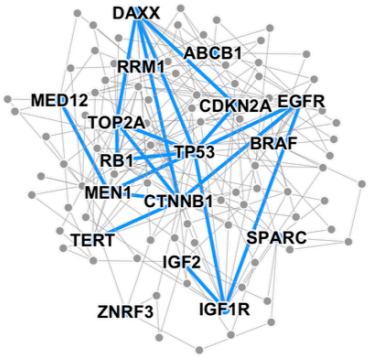


Image credit: SalientNetworks

Computer Networks



Disease Pathways

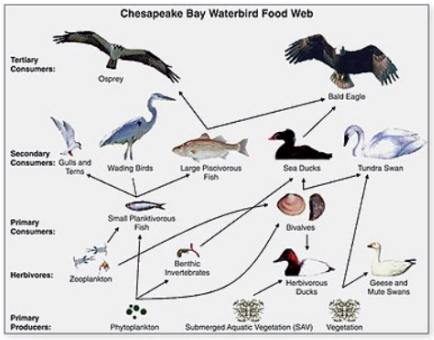


Image credit: Wikipedia

Food Webs

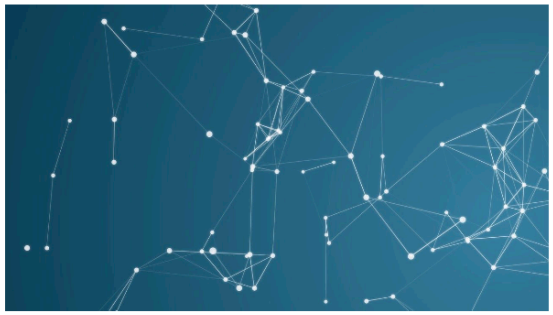


Image credit: Pinterest

Particle Networks

Image credit: visitlondon.com

Underground Networks

Many applications - Natural graphs

Image credit: Medium

Social Networks

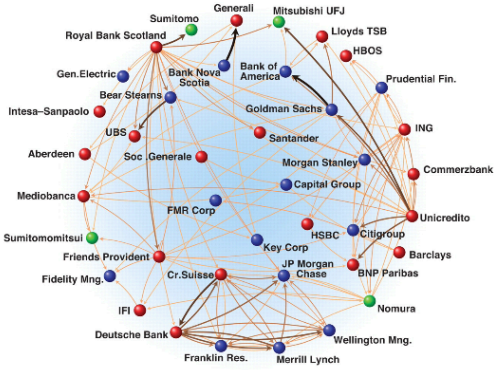


Image credit: Science

Economic Networks

Image credit: Lumen Learning

Communication Networks

Citation Networks

Image credit: Missoula Current News

Internet

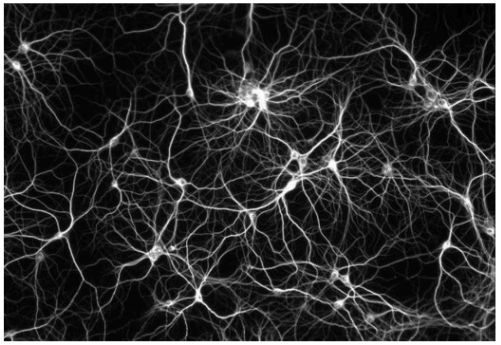


Image credit: The Conversation

Networks of Neurons

Many applications - Graph representation

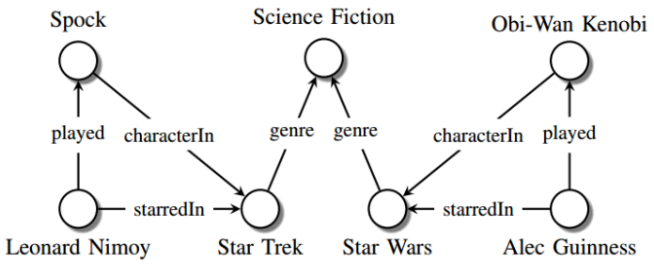


Image credit: Maximilian Nickel et al

Knowledge Graphs

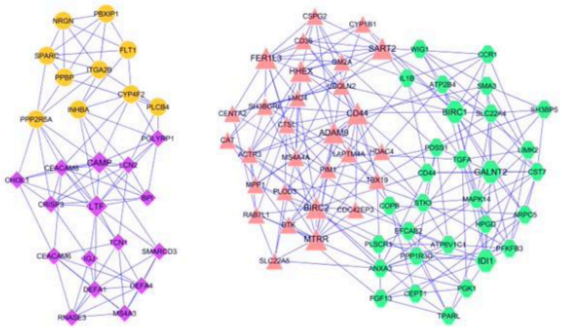


Image credit: ese.wustl.edu

Regulatory Networks

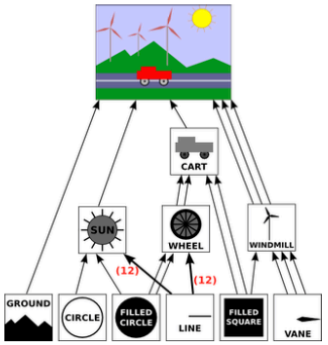


Image credit: math.hws.edu

Scene Graphs

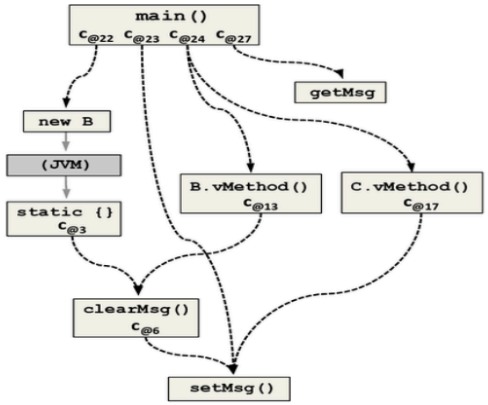


Image credit: ResearchGate

Code Graphs

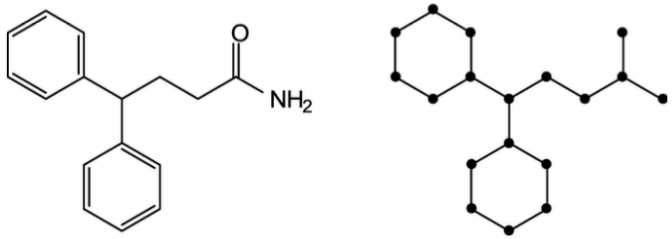


Image credit: MDPI

Molecules

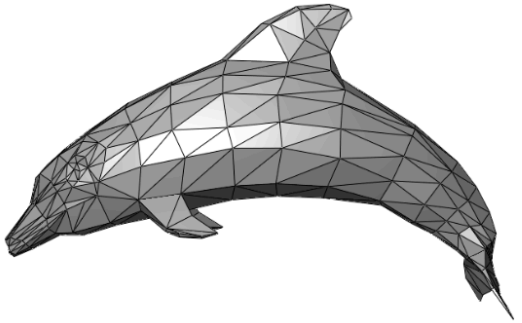


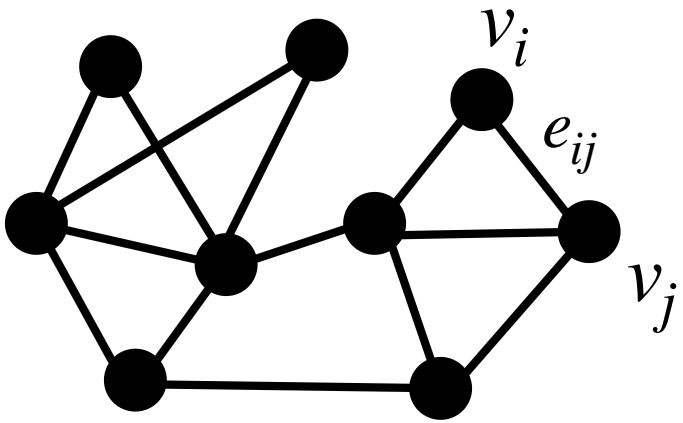
Image credit: Wikipedia

3D Shapes

Graph representation

Graphs

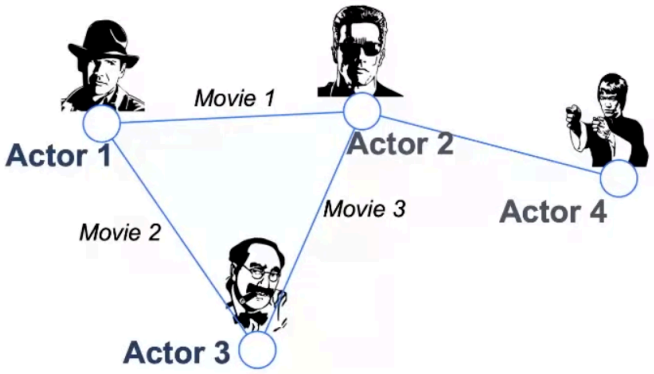
Graphs are a general language for describing and analyzing structured data with relations/ interactions

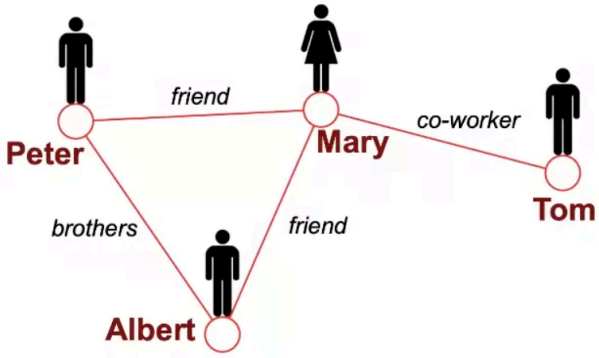


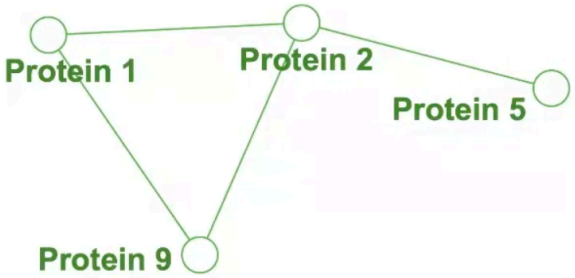
- V : Nodes, $|V| = N$
- E : Edges
- $\mathcal{N}(v_i) = \{v_j | e_{ij} \in E\}$: Neighborhood

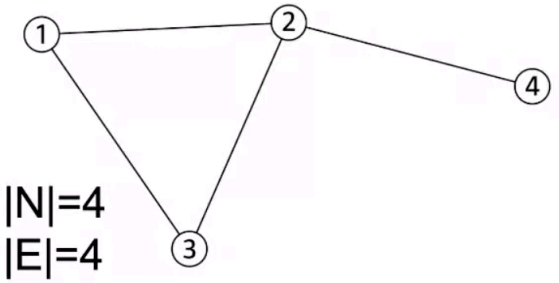
$$G = (V, E)$$

Graphs as a common language









Different domains, but same underlying mathematical representation

Choosing a proper representation

1. **Connect individuals…**
    a. **…** working together ==> professional network
    b. … living together ==> familiar network
2. **Connect papers …**
    a. **…** with same citations ==> citation network
    b. … with same authors ==> collaboration network

- In a. and b., the same nodes, different connections, with different meaning

---

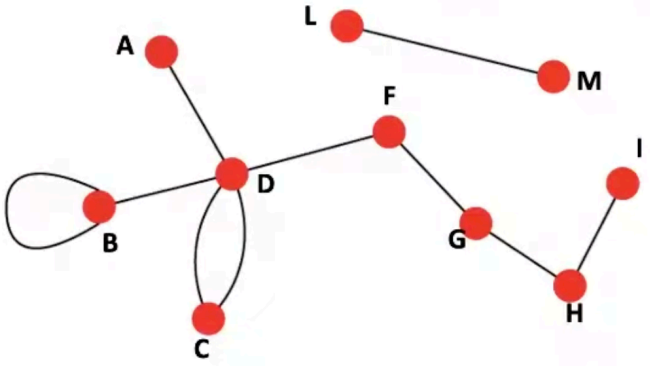
3. In **non-structural scenarios** the graph structure is not explicitly provided => choose arbitrarily (e.g. k-nn or fully connected), or Graph Structure Learning

The **connection choice** determines the nature of the question and of the prediction we will make

Directed vs Undirected graphs

Undirected

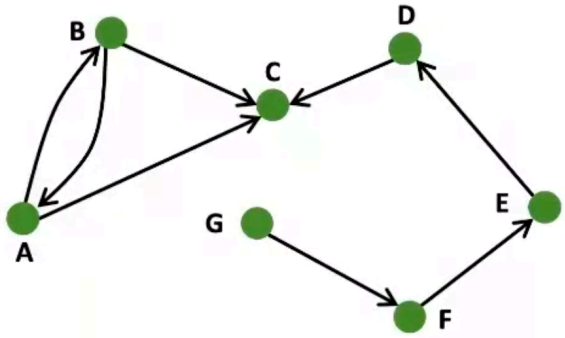
- **Links:** undirected (symmetrical, reciprocal)



- **Examples:**
  - Collaborations
  - Friendship on Facebook

Directed

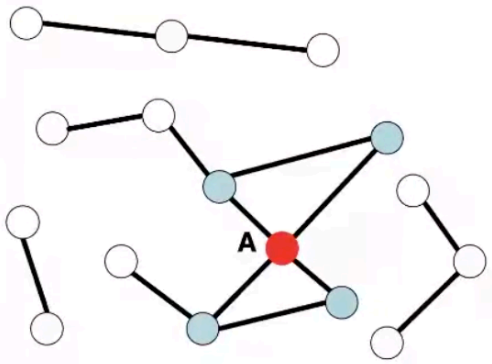
- **Links:** directed (arcs)



- **Examples:**
  - Phone calls
  - Following on Twitter

Node degree

Undirected



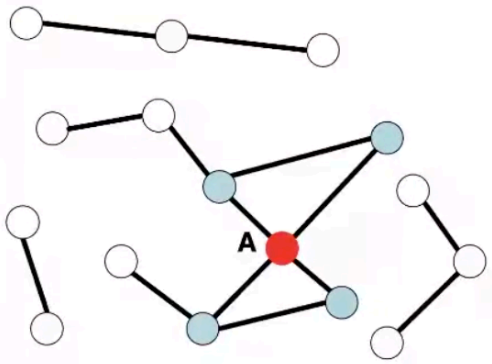
Node degree, k_i: the number of edges adjacent to node ***i***

$k_A = 4$

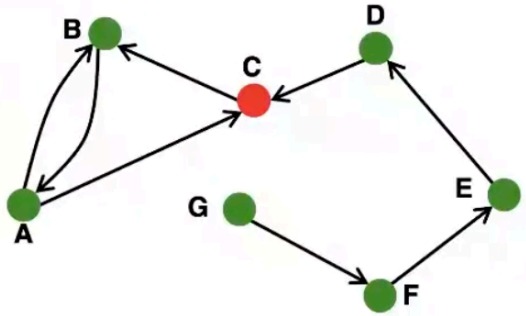
Avg. degree: $\bar{k} = \langle k \rangle = \frac{1}{N}\sum_{i=1}^{N} k_i = \frac{2E}{N}$

Node degree

Undirected



Directed



Source: Node with $k^{in} = 0$
Sink: Node with $k^{out} = 0$

Node degree, k_i: the number of edges adjacent to node ***i***

$$k_A = 4$$

Avg. degree: $\bar{k} = \langle k \rangle = \frac{1}{N}\sum_{i=1}^{N} k_i = \frac{2E}{N}$

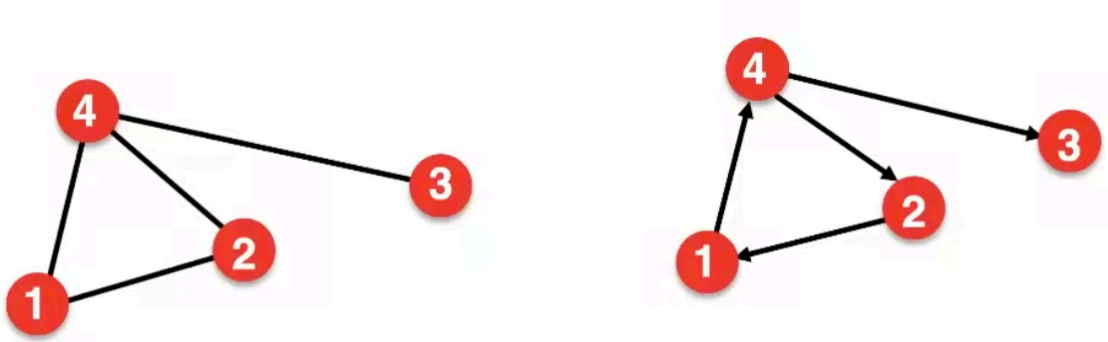
In directed networks we define an **in-degree** and **out-degree.**

The (total) degree of a node is the sum of in- and out-degrees.

$$k_C^{in} = 2 \qquad k_C^{out} = 1 \qquad k_C = 3$$

$$\bar{k} = \frac{E}{N} \qquad \overline{k^{in}} = \overline{k^{out}}$$

Graph representation: adjacency matrix



$A_{ij} = 1$ if there is a link from node i to node j

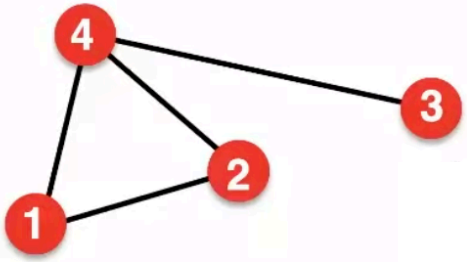
$A_{ij} = 0$ otherwise

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \qquad A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

For directed graphs
A is not symmetric

Adjacency matrix vs Node degree

Undirected



Adjacency matrix

$$A_{ij} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

$$A_{ij} = A_{ji}$$

$$A_{ii} = 0$$

Node degree

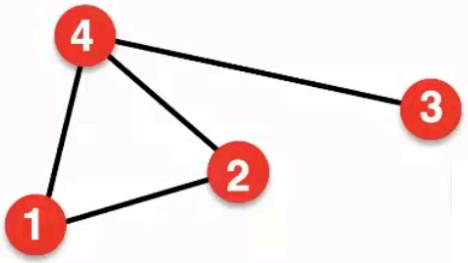
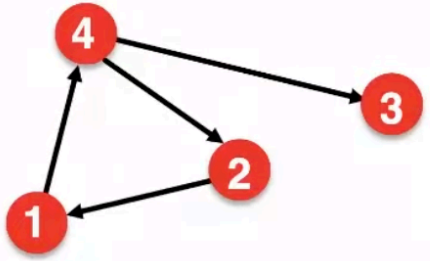
$$k_i = \sum_{j=1}^{N} A_{ij}$$

$$k_j = \sum_{i=1}^{N} A_{ij}$$

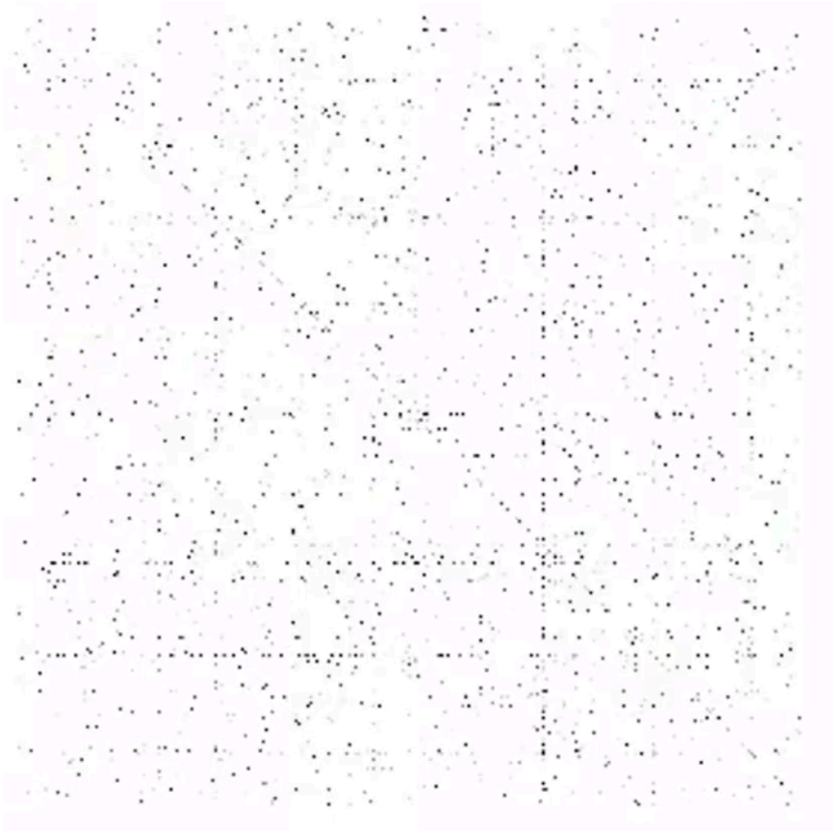
$$L = \frac{1}{2}\sum_{i=1}^{N} k_i = \frac{1}{2}\sum_{ij}^{N} A_{ij}$$

Total nb. of edges

Adjacency matrix vs Node degree

		Adjacency matrix	Node degree
Undirected		$A_{ij} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$ $A_{ij} = A_{ji}$ $A_{ii} = 0$	$k_i = \sum_{j=1}^{N} A_{ij}$ $k_j = \sum_{i=1}^{N} A_{ij}$ $L = \frac{1}{2}\sum_{i=1}^{N} k_i = \frac{1}{2}\sum_{ij} A_{ij}$ Total nb. of edges
Directed		$A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$ $A_{ij} \neq A_{ji}$ $A_{ii} = 0$	$k_i^{out} = \sum_{j=1}^{N} A_{ij}$ $k_j^{in} = \sum_{i=1}^{N} A_{ij}$ $L = \sum_{i=1}^{N} k_i^{in} = \sum_{j=1}^{N} k_j^{out} = \sum_{i,j}^{N} A_{ij}$ Total nb. of edges

Adjacency matrix of real world example

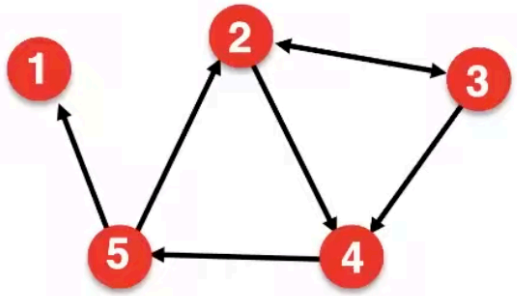


extremely sparse!

represented as a sparse matrix

Graph representation: edge list

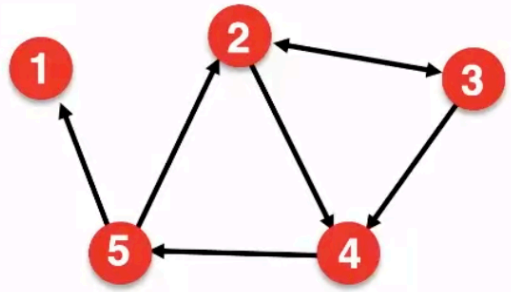
- **Represent graph as a list of edges:**
  - (2, 3)
  - (2, 4)
  - (3, 2)
  - (3, 4)
  - (4, 5)
  - (5, 2)
  - (5, 1)



Hard to do matrix analysis and manipulation

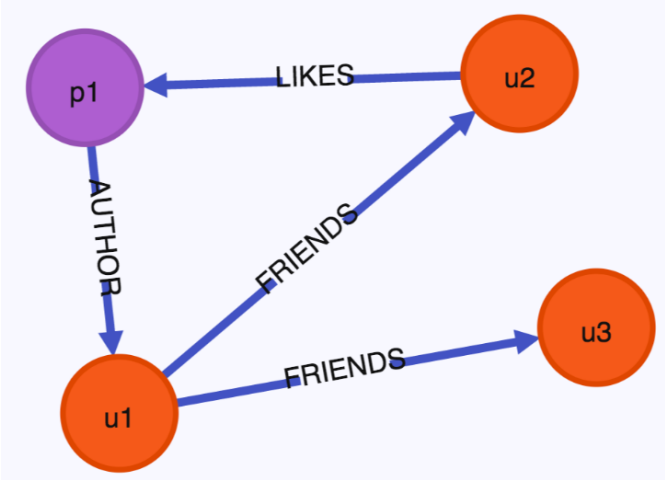
Graph representation: adjacency list

- **Adjacency list:**
  - Easier to work with if network is
    - Large
    - Sparse
  - Allows us to quickly retrieve all neighbors of a given node
    - 1:
    - 2: 3, 4
    - 3: 2, 4
    - 4: 5
    - 5: 1, 2

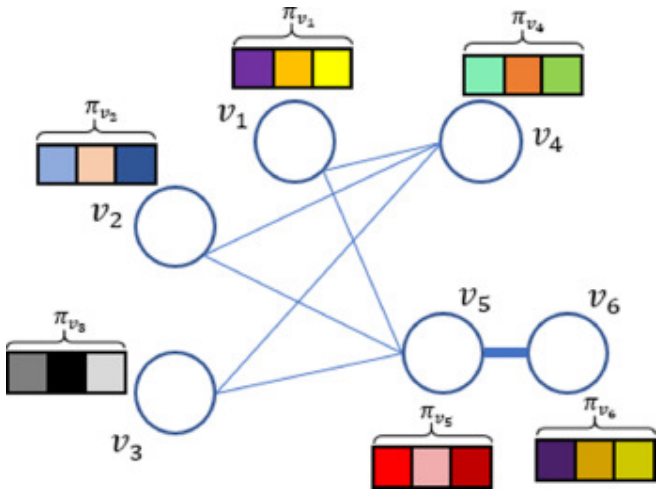


Node and edge attributes

- Possible **attributes for edges**:
  - **Weight** (e.g., frequency of communication)
  - **Ranking** (best friend, second best friend…)
  - **Type** (friend, relative, co-worker)
  - **Sign**: Friend vs. Foe, Trust vs. Distrust



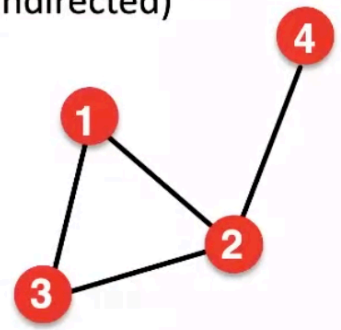
- Possible **attributes for nodes**:
  - **age, gender, job** for individuals
  - **date, journal, authors** for papers



Weighted vs Unweighed edges

- **Unweighted**

(undirected)



$$A_{ij} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

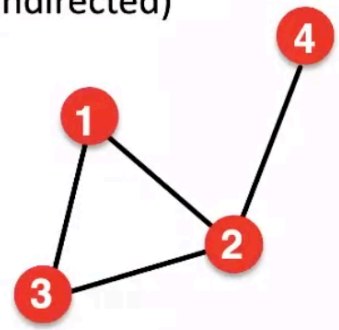
$$A_{ii} = 0 \qquad A_{ij} = A_{ji}$$

$$E = \frac{1}{2}\sum_{i,j=1}^{N} A_{ij} \qquad \bar{k} = \frac{2E}{N}$$

Examples: Friendship, Hyperlink

Weighted vs Unweighed edges

- **Unweighted**
  (undirected)



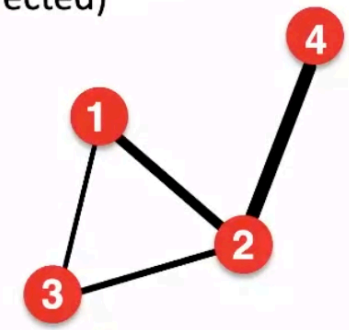
$$A_{ij} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

$$A_{ii} = 0 \qquad A_{ij} = A_{ji}$$

$$E = \frac{1}{2}\sum_{i,j=1}^{N} A_{ij} \qquad \bar{k} = \frac{2E}{N}$$

Examples: Friendship, Hyperlink

- **Weighted**
  (undirected)



$$A_{ij} = \begin{pmatrix} 0 & 2 & 0.5 & 0 \\ 2 & 0 & 1 & 4 \\ 0.5 & 1 & 0 & 0 \\ 0 & 4 & 0 & 0 \end{pmatrix}$$

$$A_{ii} = 0 \qquad A_{ij} = A_{ji}$$

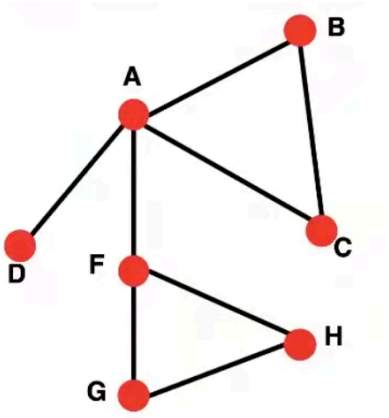
$$E = \frac{1}{2}\sum_{i,j=1}^{N} nonzero(A_{ij}) \qquad \bar{k} = \frac{2E}{N}$$

Examples: Collaboration, Internet, Roads

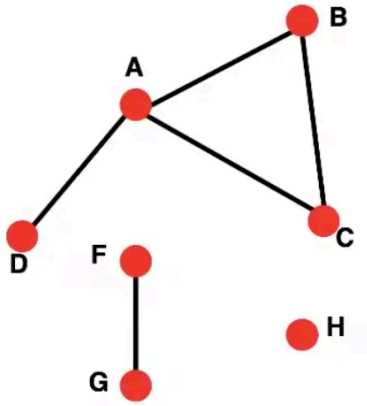
Connectivity of Undirected graphs

- **Connected** (undirected) graphs:
  - any pair of nodes can be joined by a path

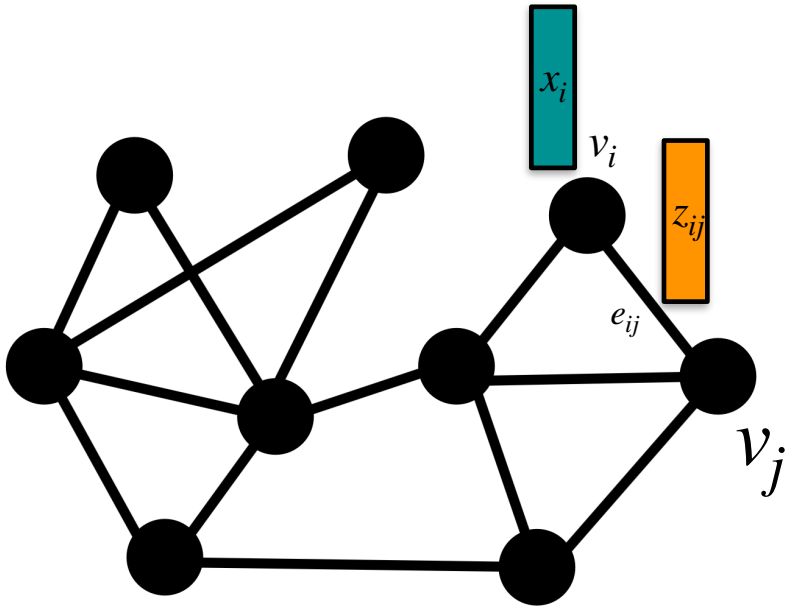
Connected



Unconnected



Graphs



$G = (V, E)$

- V : nodes, $|V| = N$
- E : edges
- $\mathcal{N}(v_i) = \{v_j | e_{ij} \in E\}$: Neighborhood

- $A, |A| = N \times N$: Adiacency matrix
  $A(i,j) = 1$ iff $e_{ij} \in E$ else $A(i,j) = 0$

- $x_i \in R^D$: feature *i-th* node
  $X, |X| = N \times D$: collection of node features

- $z_{ij} \in R^F$: feature *of the edge* e_{ij}

Graph: Machine Learning

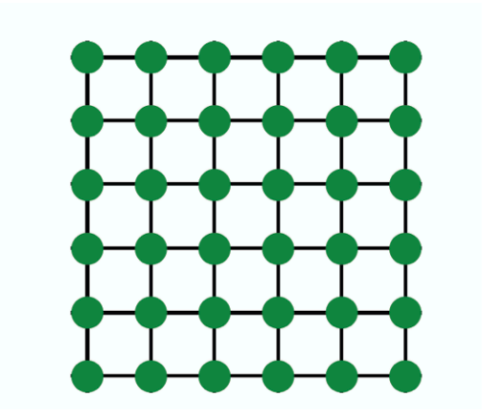
RECAP: Complex domains have a rich relational structure, which can be represented as a relational graph

RECAP: By explicitly modeling relationships we achieve better performance

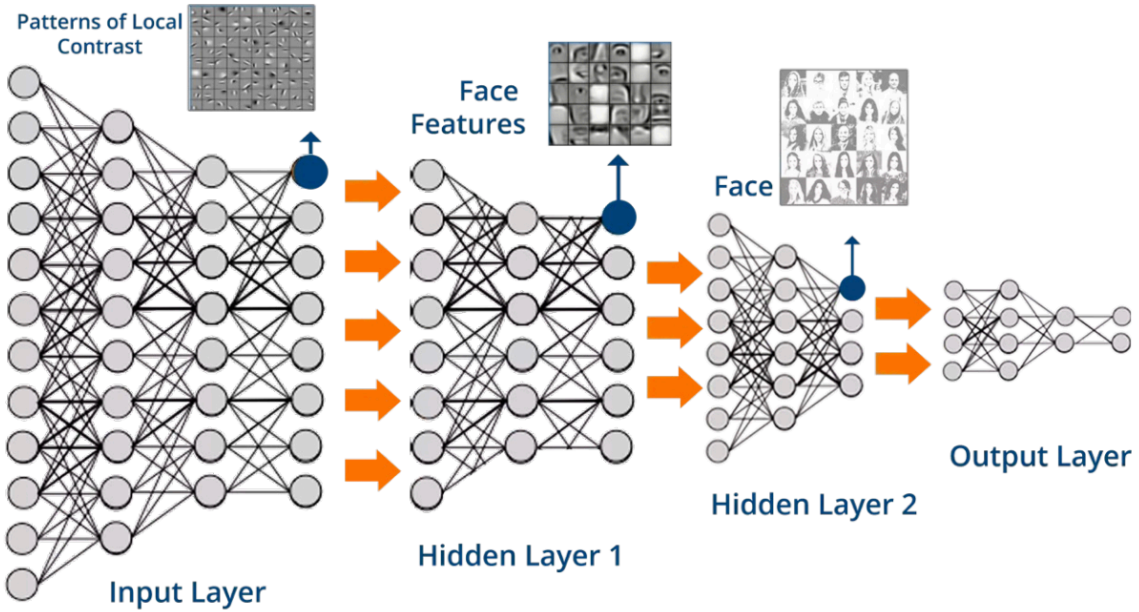
Main question:

How do we take advantage of relational structure for better prediction?

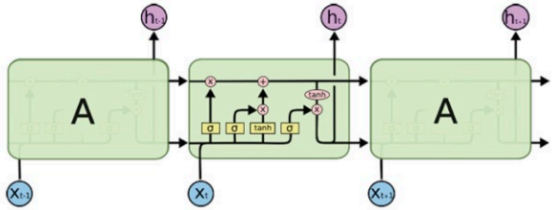
Deep Learning architecture



Images



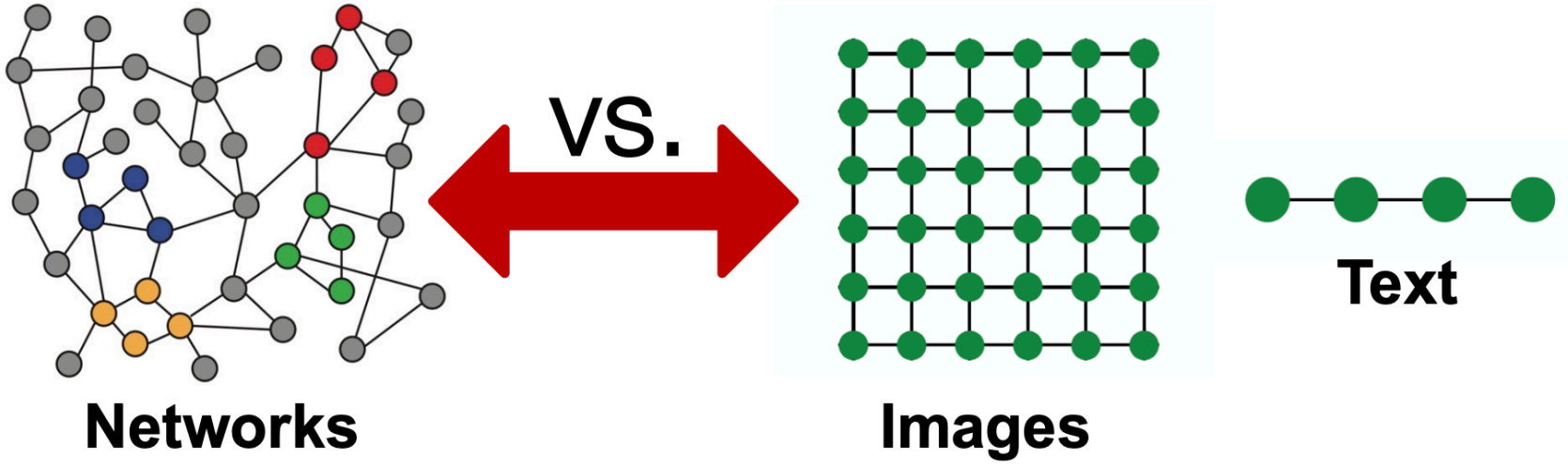
Text/Speech



Modern deep learning toolbox is designed for structured data: sequences or grids

Networks are complex

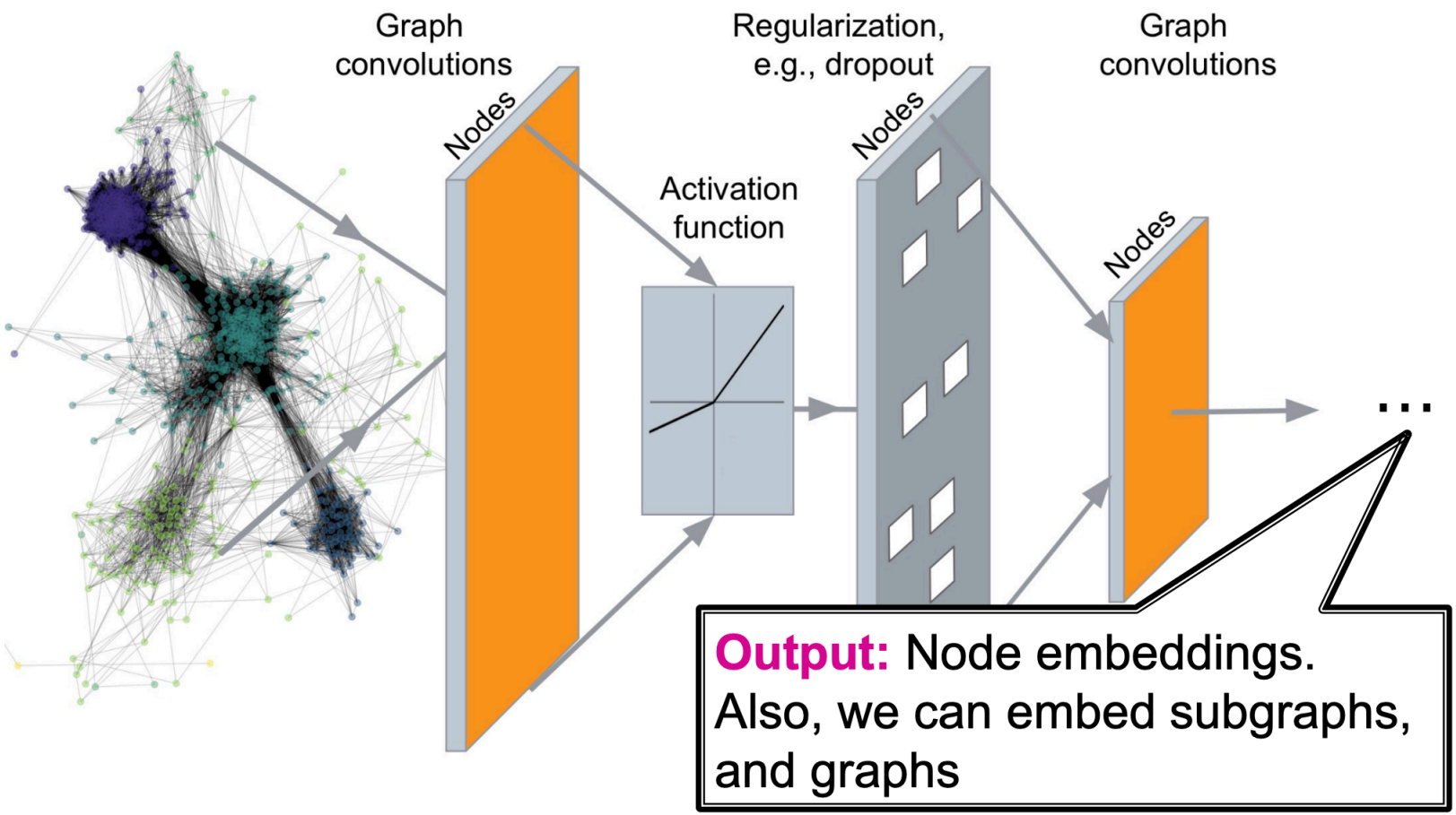
- Arbitrary size and complex topological structure (i.e., no spatial locality like grids)
- No fixed node ordering or reference point
- No fixed-length inputs (e.i. neighborhood could change dimensionality for each node)
- Often dynamic and have multimodal features



Graphs on Machine Learning

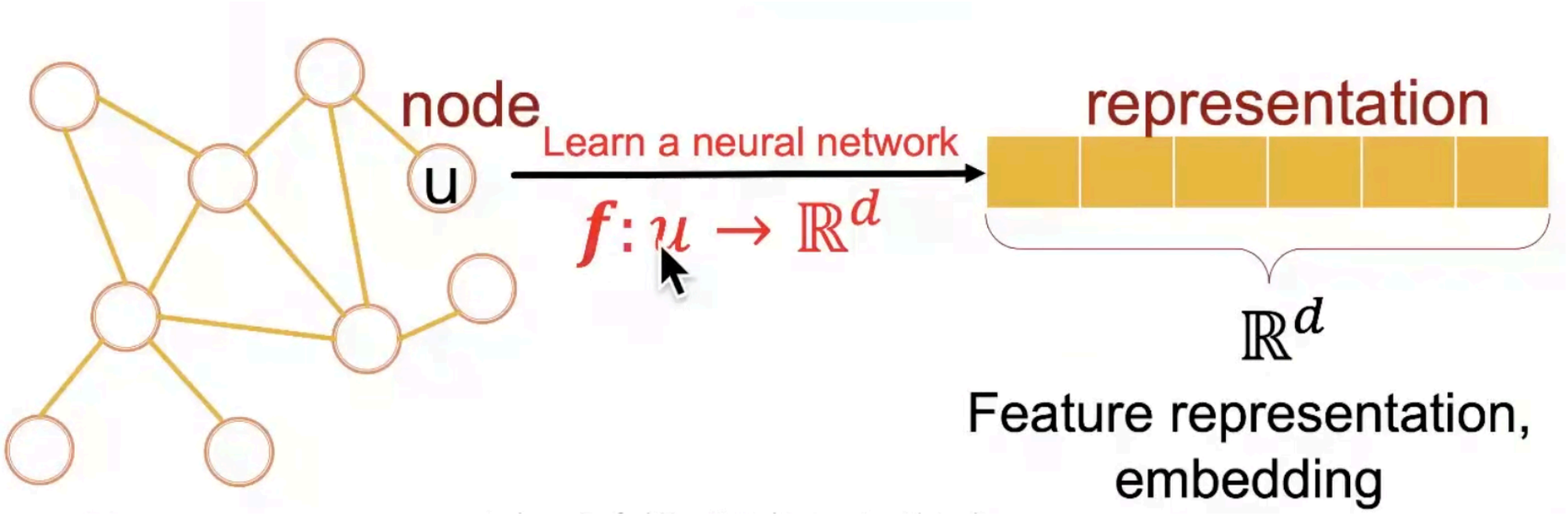
Graphs on Machine Learning

What we want:



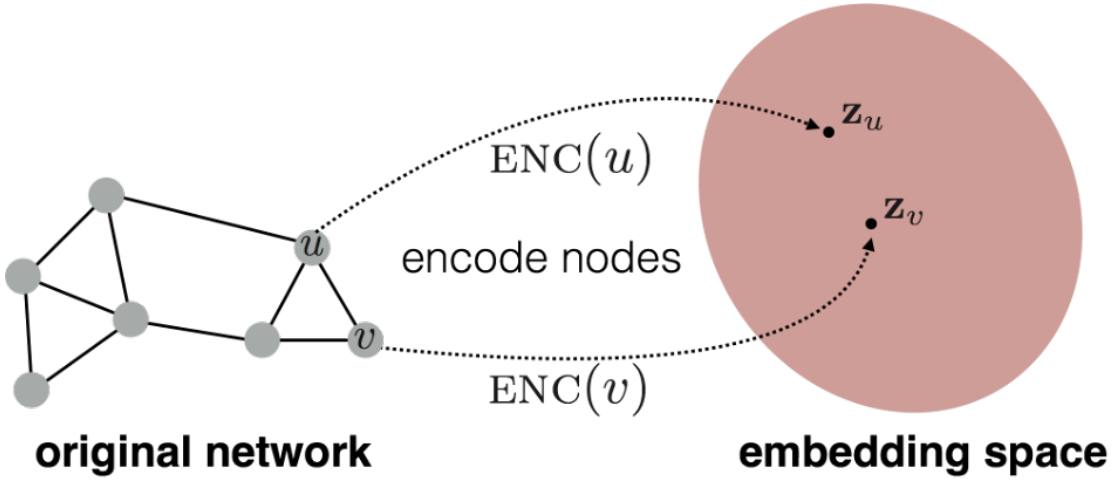
Representation learning

Map nodes to d-dimensional **embeddings** …



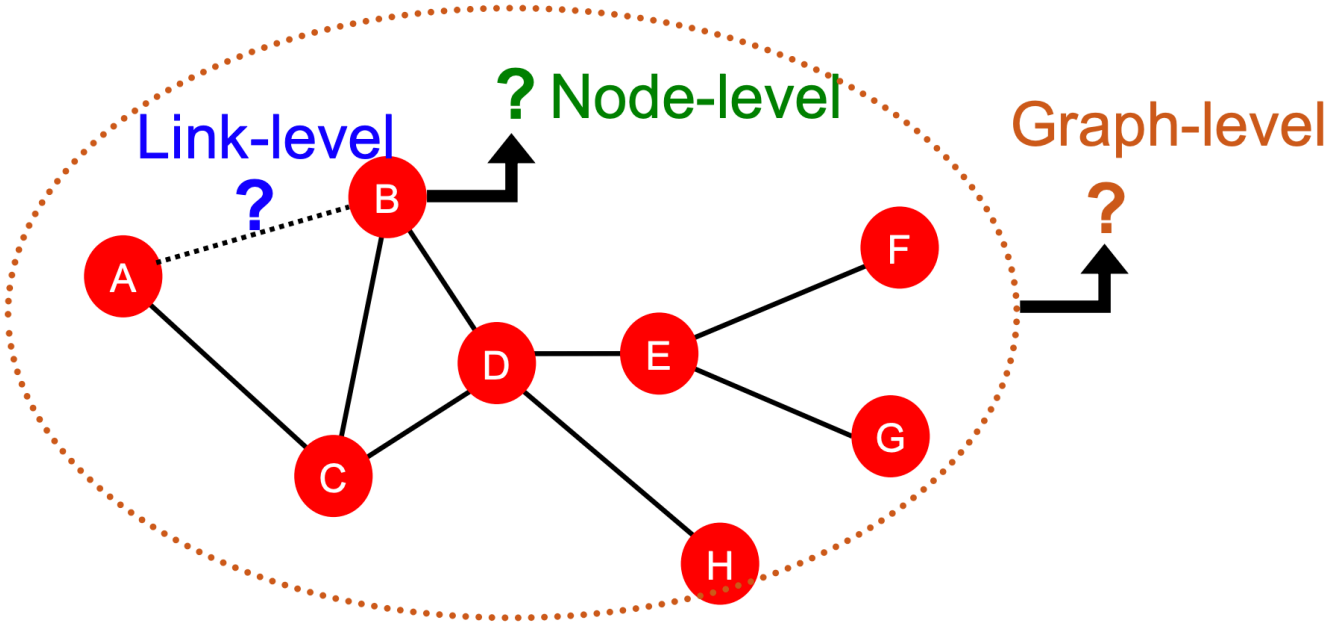
Representation learning

… such that **similar nodes** in the network are embedded **close together**



Applications

- node-level prediction
- link-level prediction
- graph-level prediction



Applications

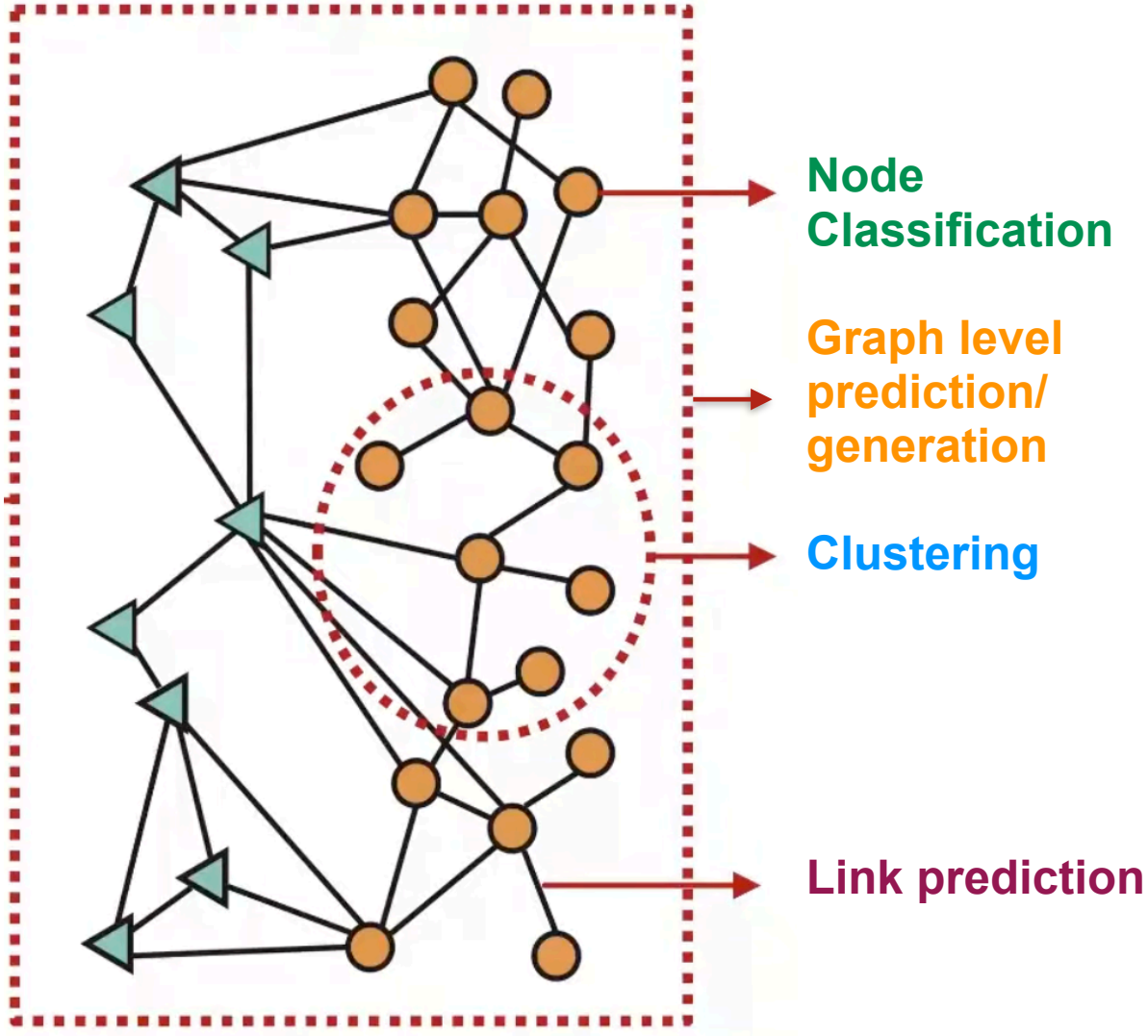
- **Node classification:** Predict a property of a node
  Example: Categorize online users / items

- **Link prediction:** Predict whether there are missing links between two nodes
  Example: Knowledge graph completion

- **Graph classification:** Categorize different graphs
  Example: Molecule property prediction

- **Clustering:** Detect if nodes form a community
  Example: Social circle detection

- **Other tasks:**
  - Graph generation: Drug discovery
  - Graph evolution: Physical simulation



Permutation invariance/equivariance

Being P : permutation matrix

- Node features are permute by $\tilde{X} = PX$
- Adiacency matrix is permute by $\tilde{A} = PAP^T$

P has the same dimensions as A, contains a single 1 in each row and in each column, s.t a 1 in position (i, j) means that node i and node j permute between each other

- **permutation-invariant**: $y(A, X) = y(\tilde{A}, \tilde{X})$

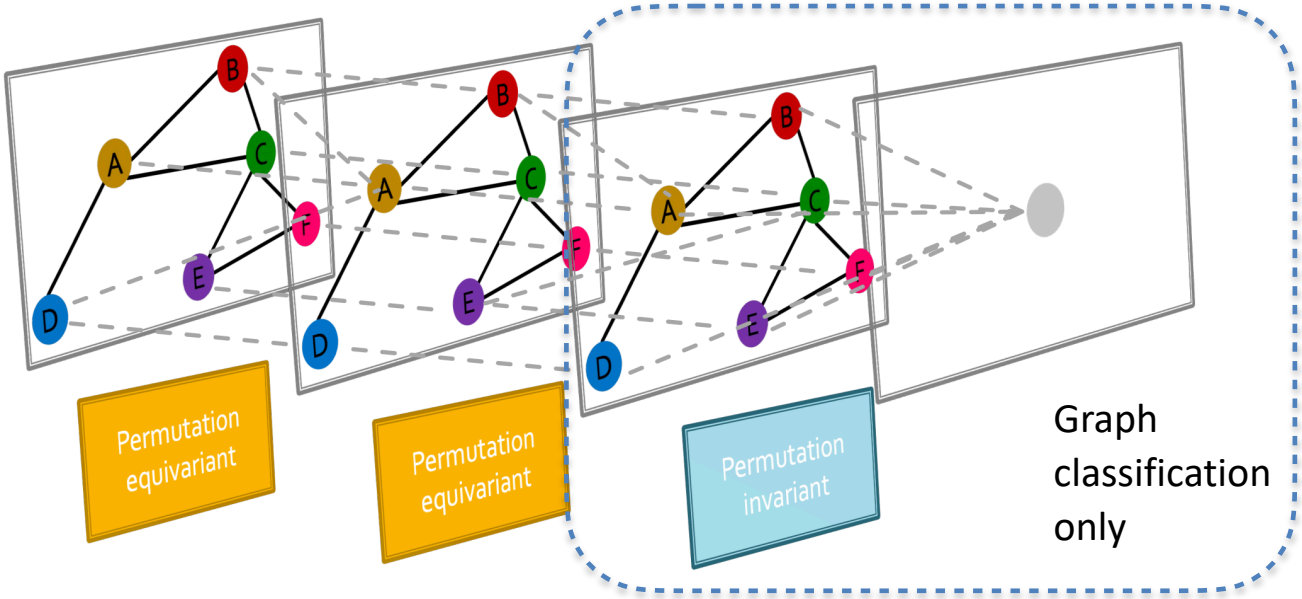
Network prediction must be invariant to node label reordering
[$y(\,.\,,.\,)$ output of the network]

- **permutation-equivariance**: $\mathbf{y}(\tilde{X}, \tilde{A}) = P\mathbf{y}(X, A)$

Node prediction must be equivariant wrt node label reordering
[$\mathbf{y}(\,.\,,.\,)$ output of a node]

Graphs on machine learning

- Graph neural networks consist of multiple permutation equivariant, and for graph classification with a final invariant layer

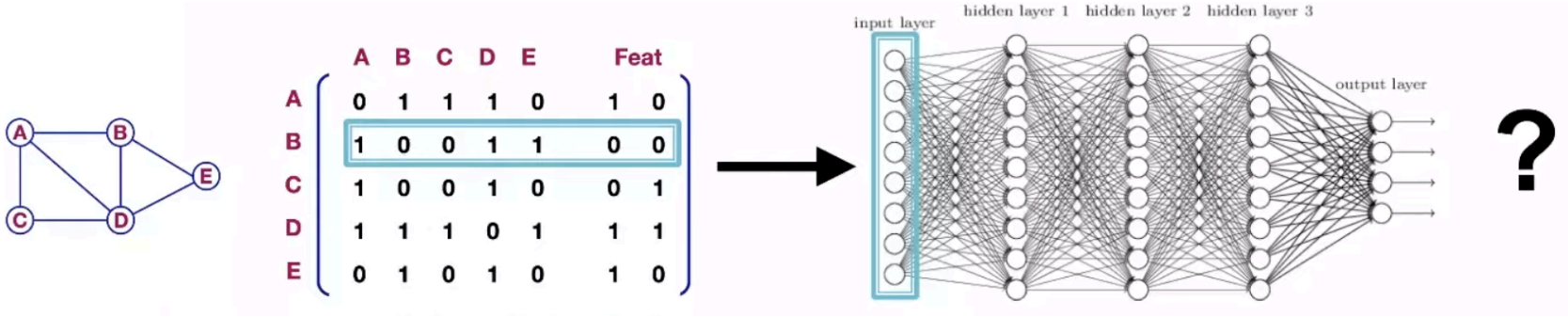


To input a graph into a DL model, we must fix an ordering, but this order is **arbitrary**, so the output must not depend on it.

- For **node/edge classification**, layers must be **permutation equivariant**.
- For **graph classification**, we need a **permutation-invariant** layer.

Deep learning introduced so far: MLP

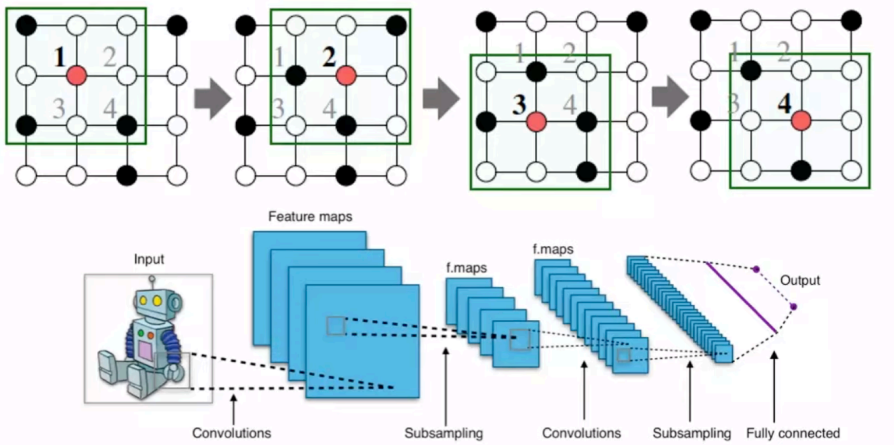
- Try to apply MPL to graphs:
  - Join Adjacency matrix and features
  - feed each row as a training example to a MLP



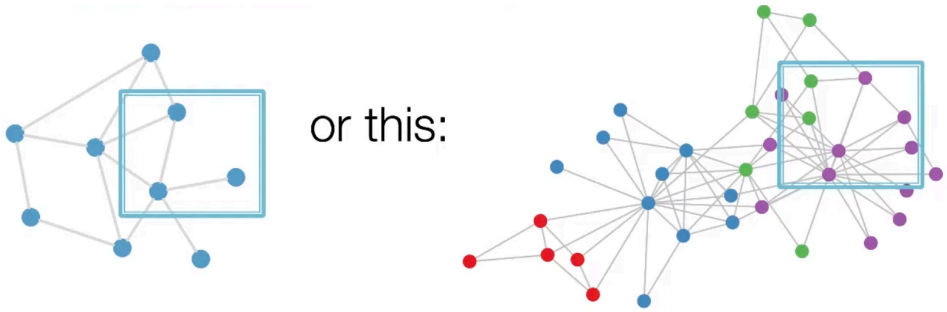
- Issues of this idea:
  - $O(|V|)$ parameters
  - Not applicable to graphs of different size
  - Sensitive to node order

Deep learning introduced so far: CNN

- Try to apply CNN to graphs:



- **Issues of this idea:**
  - with graphs there is **no fixed notation of locality** (window dimension) or sliding window
  - still **not permutation invariant**

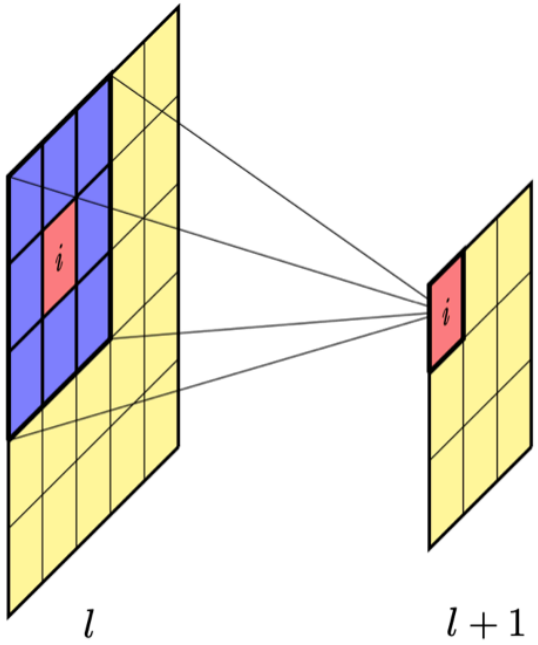


Need something new

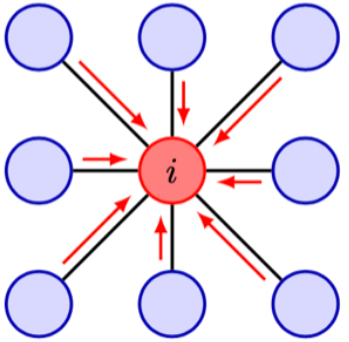
Convolutional filters

- **Let maintain what is good in convolutional filter:**
  - collect the information from the neighbors
  - and combine it

- **But we need to modify it to obtain:**
  1. Permutation invariance
  2. Flexibility on the number of neighbors



$$z_i^{(l+1)} = f\left(\sum_j w_j z_j^{(l)} + b\right)$$

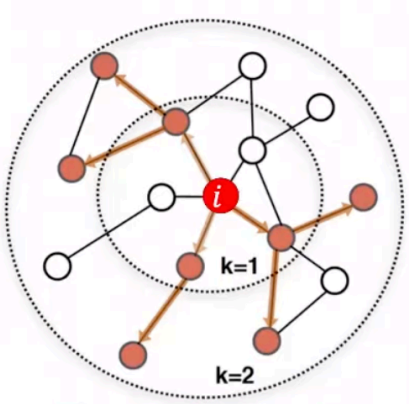


Let's start viewing
the filter as a graph.

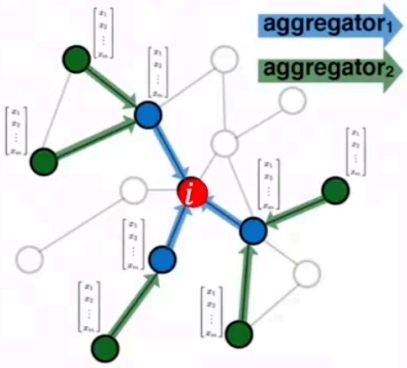
Graph Convolutional Network

Issue: in a graph we do not have a regular grid, and a fix number of neighbors

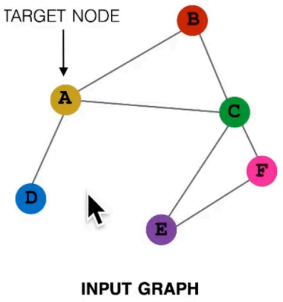
Solution: exploit node's neighborhood to define the computation graph

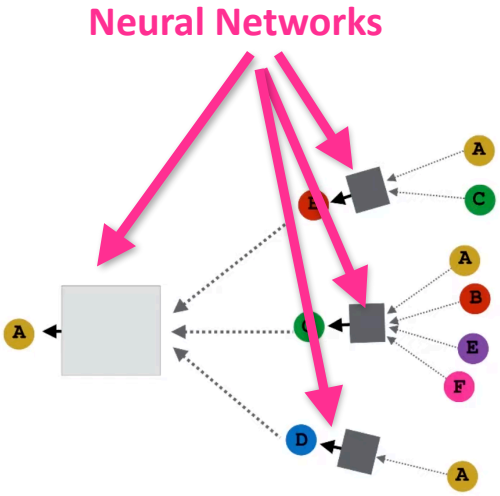


Determine node computation graph



Propagate and transform information



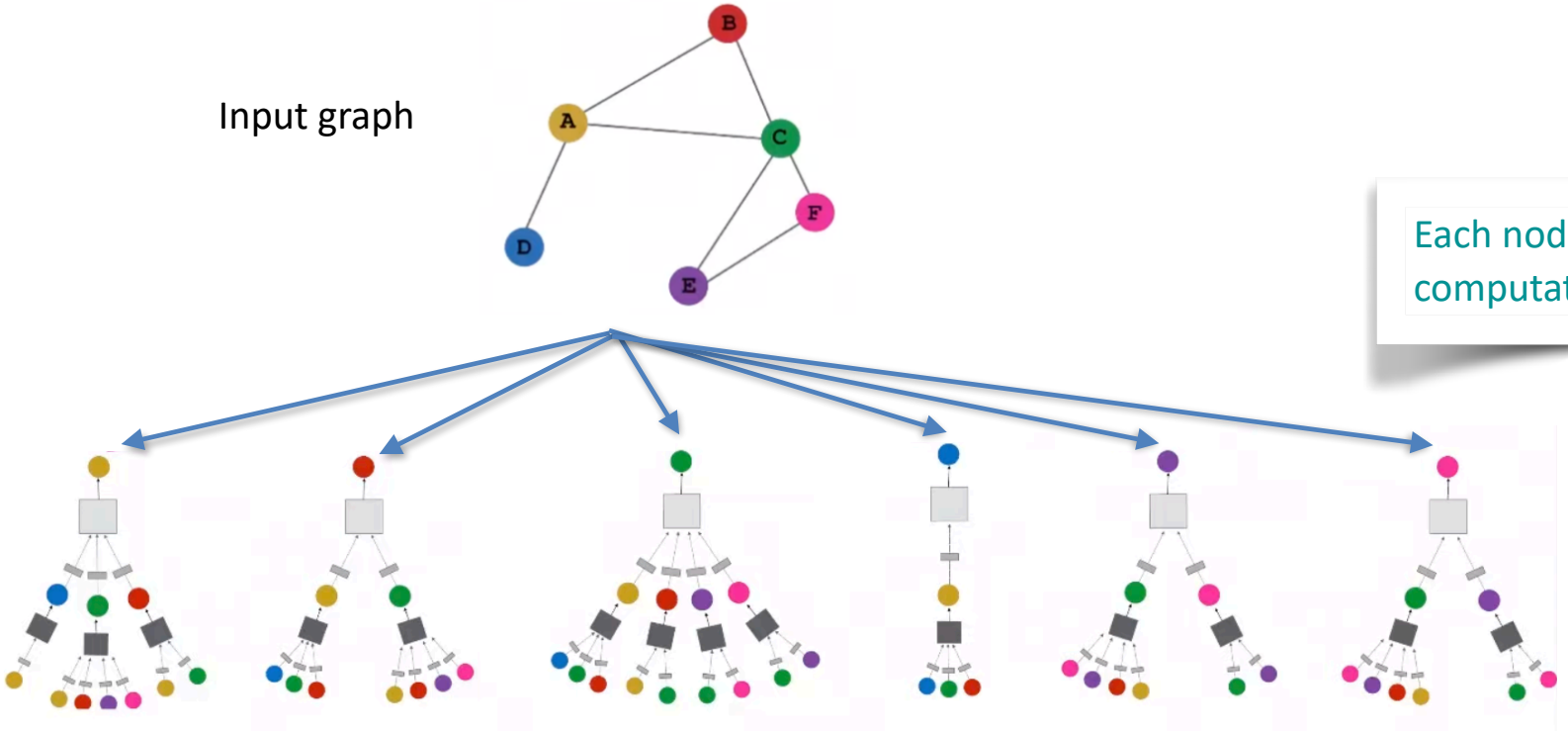


Nodes **aggregate** information from their neighbors and **process** them using neural networks

Idea: Aggregate Neighbors

Interesting:

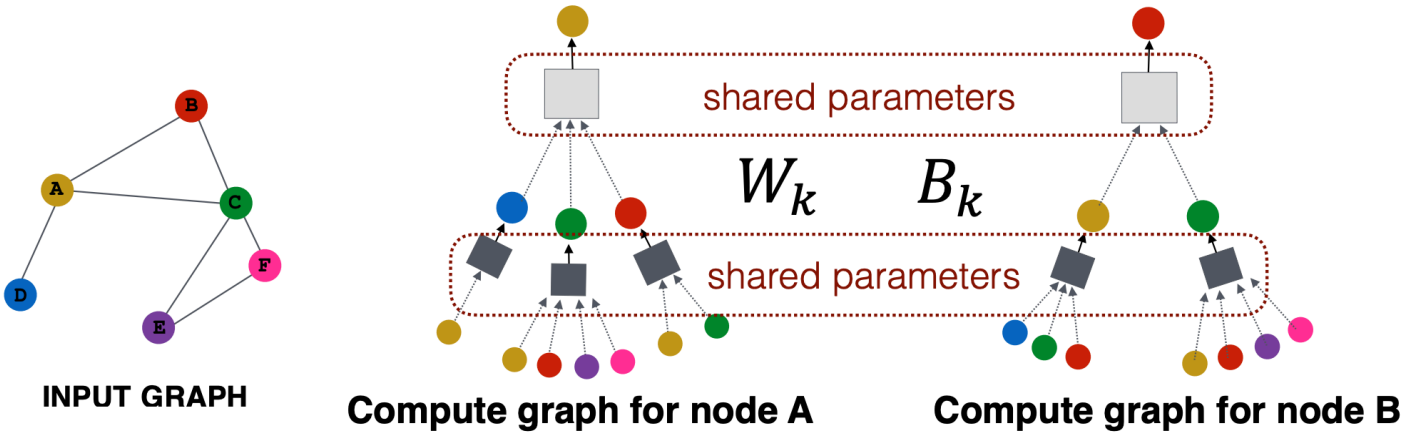
- every node defines its computation graph
- we are going to learn on multiple architecture simultaneously



Each node has a different computation graph!

Inductive capability

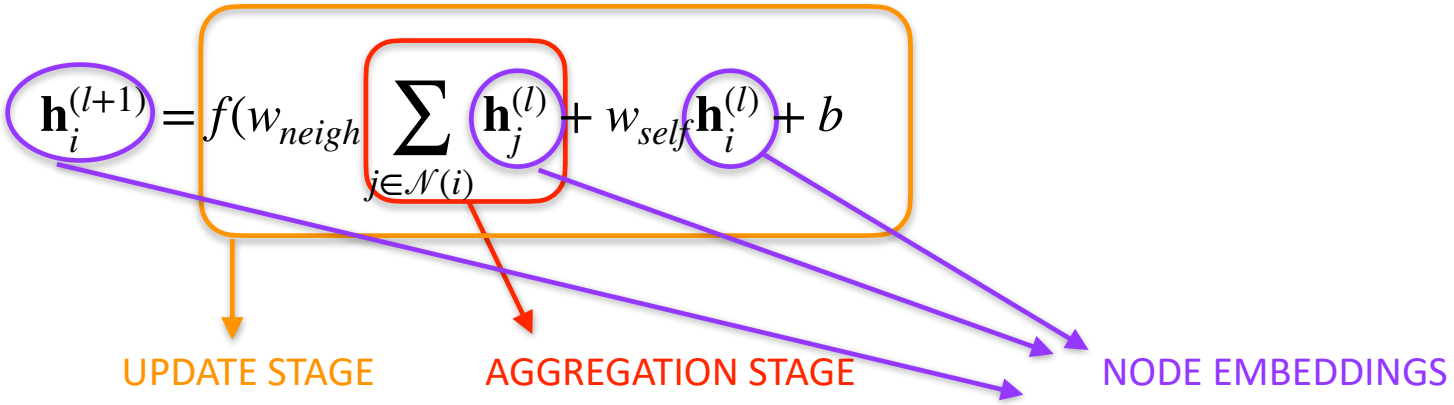
- The same aggregation parameters are **shared** for all nodes at the same layer k:
  - The number of model parameters is sublinear in $|V|$ and we can generalize to unseen nodes!



Graph convolutional layer

Mathematically:

- build it on the convolutional filters
- nonlinear transformation of node embeddings differentiable wrt weights



- **Summation** is an operator flexible to the number of neighbors
- **Weight** is applied AFTER summation so it is permutation invariant

Message-passing neural network

- Being $\mathbf{h}_n^{(l)} \in R^D$ the embedding of node n at layer l, we define:

- Aggregation STEP:

$$\mathbf{z}_n^{(l)} = \text{Aggregate}(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\})$$

- Update STEP:

$$\mathbf{h}_n^{(l+1)} = \text{Update}(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)})$$

Message-passing neural network

Algorithmically:

Input: Undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
Initial node embeddings $\{\mathbf{h}_n^{(0)} = \mathbf{x}_n\}$
Aggregate$(\cdot)$ function
Update$(\cdot, \cdot)$ function

Output: Final node embeddings $\{\mathbf{h}_n^{(L)}\}$

```
// Iterative message-passing
```

for $l \in \{0, \ldots, L-1\}$ **do**

$$\mathbf{z}_n^{(l)} \leftarrow \text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right)$$

$$\mathbf{h}_n^{(l+1)} \leftarrow \text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right)$$

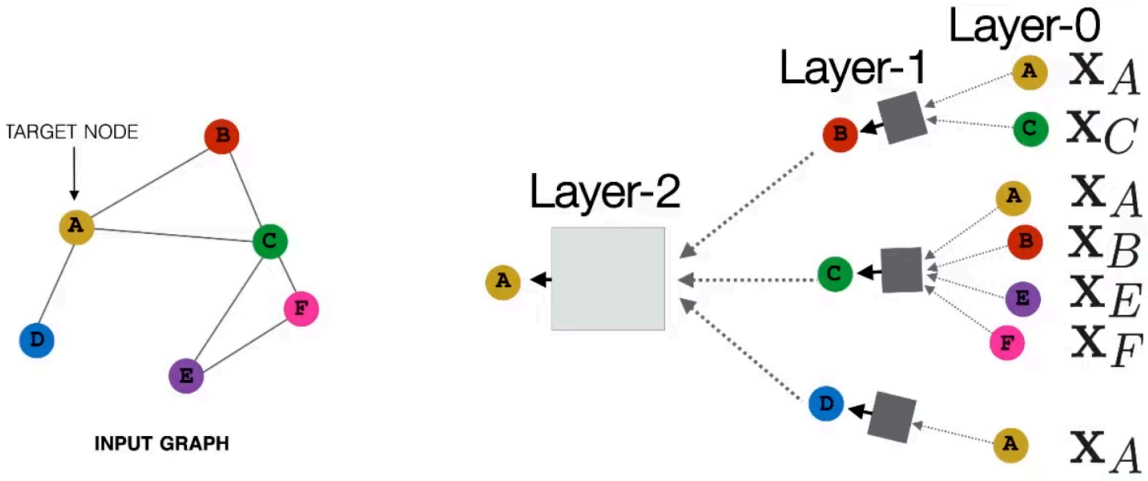
end for

return $\{\mathbf{h}_n^{(L)}\}$

Graph models: many layers

Model can be of **arbitrary depth:**

- Nodes have embeddings at each layer
- Layer-0: embedding of node u is its input feature, x_u
- Layer-k: embedding gets information from nodes that are K hops away



Aggregator operators

- Recall: $\mathbf{z}_n^{(l)} = \text{Aggregate}(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\})$
- Basic aggregation function: the sum

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}$$

The sum is influenced by the cardinality of the neighborhood

- Other aggregation functions face this problem:

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \frac{1}{|\mathcal{N}(n)|} \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}$$

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} \frac{\mathbf{h}_m^{(l)}}{\sqrt{|\mathcal{N}(n)|\,|\mathcal{N}(m)|}}$$

Symmetric Normalization: assigning a lower weight to nodes with a high degree because they are less informative (linked to everyone...).

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \max(\mathbf{h}_m^{(l-1)}), m \in \mathcal{N}(n)$$

Aggregator operators with training GraphSAGE variant

Learnable weight matrix

$$\mathbf{h}_v^k = \sigma\left(\left[\mathbf{A}_k \cdot \text{AGG}(\{\mathbf{h}_u^{k-1}, \forall u \in N(v)\}), \mathbf{B}_k \mathbf{h}_v^{k-1}\right]\right)$$

AGG: any differentiable, and permutation invariant function that maps set of vectors (embeddings) to a single vector.

Concatenate self embedding and neighbor embedding

GraphSAGE, AGGRETATION Variants

- Mean: $$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$$

GraphSAGE, AGGRETATION Variants

- Mean: $$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$$
- Pool: Transform neighbor vectors and apply symmetric vector function.

$$\text{AGG} = \boxed{\gamma}\left(\{\mathbf{Q}\mathbf{h}_u^{k-1}, \forall u \in N(v)\}\right)$$

element-wise mean/max

GraphSAGE, AGGRETATION Variants

- Mean: $$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$$
- Pool: Transform neighbor vectors and apply symmetric vector function.

$$\text{AGG} = \boxed{\gamma}\left(\{\mathbf{Q}\mathbf{h}_u^{k-1}, \forall u \in N(v)\}\right)$$

element-wise mean/max

- LSTM: Apply LSTM to random permutation of neighbors.

$$\text{AGG} = \text{LSTM}\left([\mathbf{h}_u^{k-1}, \forall u \in \pi(N(v))]\right)$$

GraphSAGE, AGGRETATION Variants

- Mean: $\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$
- Pool: Transform neighbor vectors and apply symmetric vector function.

$$\text{AGG} = \boxed{\gamma}\left(\{\mathbf{Q}\mathbf{h}_u^{k-1}, \forall u \in N(v)\}\right)$$

element-wise mean/max

- LSTM: Apply LSTM to random permutation of neighbors.

$$\text{AGG} = \text{LSTM}\left([\mathbf{h}_u^{k-1}, \forall u \in \pi(N(v))]\right)$$

- MLP: Apply an arbitrary deep multi-layer perceptron parametrized by trainable parameters $\boldsymbol{\theta}$

$$AGG = MLP_\theta\left(\sum_{u \in N(v)} MLP_\phi\ (h_u^{k-1})\right)$$

Update Operators, basic

Recall: $\mathbf{h}_n^{(l+1)} = \text{Update}(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)})$

$$\text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right) = f\left(\mathbf{W}_{\text{self}}\mathbf{h}_n^{(l)} + \mathbf{W}_{\text{neigh}}\mathbf{z}_n^{(l)} + \mathbf{b}\right)$$

OR:

$$\text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right) = f\left(\mathbf{W}_{\text{neigh}} \sum_{m \in \mathcal{N}(n), n} \mathbf{h}_m^{(l)} + \mathbf{b}\right)$$

Where:

- $\mathbf{h}_n$ is the embedding of node n, and $\mathbf{z}_n$ is define by the aggregator,
- $f(\cdot)$ is a non linear function,
- W_{self}, W_{neigh}, $\mathbf{b}$ are trainable parameters

Update Operators, GraphSAGE variants

- CONCATENATION:

$$\mathbf{h}_n^{(l+1)} = \text{Update}_{Concat}(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}) = [\text{Update}(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}) \oplus \mathbf{h}_n^{(l)}]$$

$\oplus$: concatenation

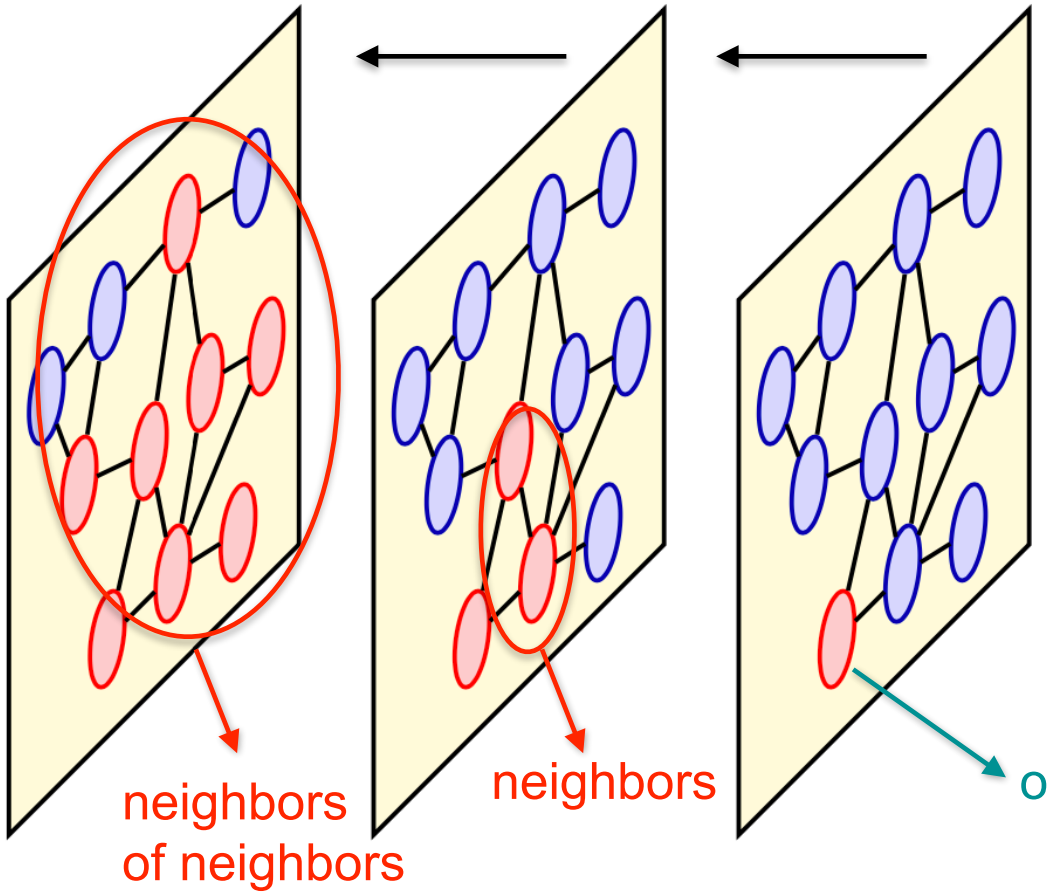
- LINEAR INTERPOLATION:

$$\mathbf{h}_n^{(l+1)} = \text{Update}_{Interp}(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}) = \alpha_1 \odot \text{Update}(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}) + \alpha_2 \odot \mathbf{h}_n^{(l)}]$$

$\odot$: element-wise multiplication

where $\alpha_1, \alpha_2 \in [0,1], \alpha_2 = 1 - \alpha_1,$ to be learnt

Receptive field

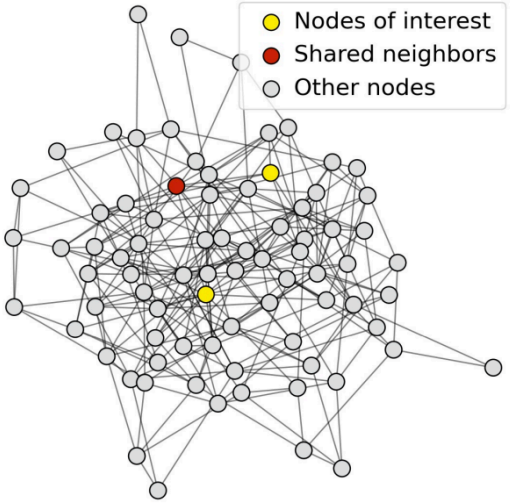


The effective receptive field, corresponding to the number of nodes shown in red, **grows with the number of processing layers**

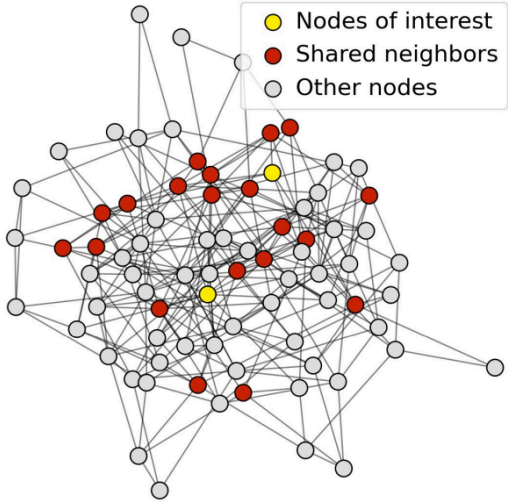
Receptive field

- **Receptive field overlap** for two nodes
  - **The shared neighbors quickly grows** when we increase the number of hops (num of GNN layers)

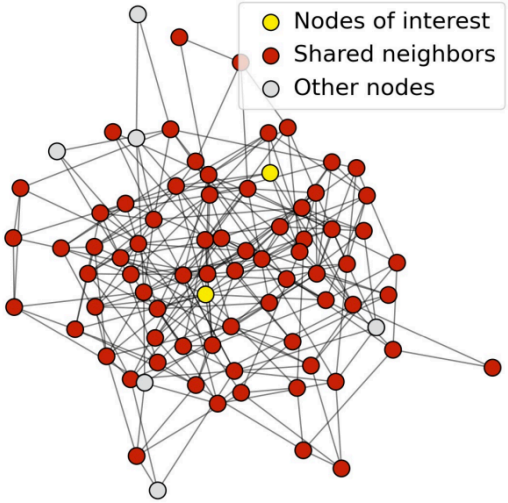
1-hop neighbor overlap
Only 1 node



2-hop neighbor overlap
About 20 nodes



3-hop neighbor overlap
Almost all the nodes!



Receptive field and GNN

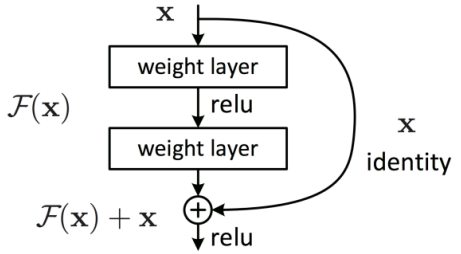
- We can explain **over-smoothing** via the notion of the receptive field
  - We know the embedding of a node is determined by its receptive field
    - **If two nodes have highly-overlapped receptive fields, then their embeddings are highly similar**
  - Stack many GNN layers ⇒ nodes will have highly-overlapped receptive fields ⇒ node embeddings will be highly similar ⇒suffer from the **over-smoothing problem**
- To prevent over-smoothing: **Residual-connection**

GCN with Residual Connection

- **A standard GCN layer**

- $\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}\right)$

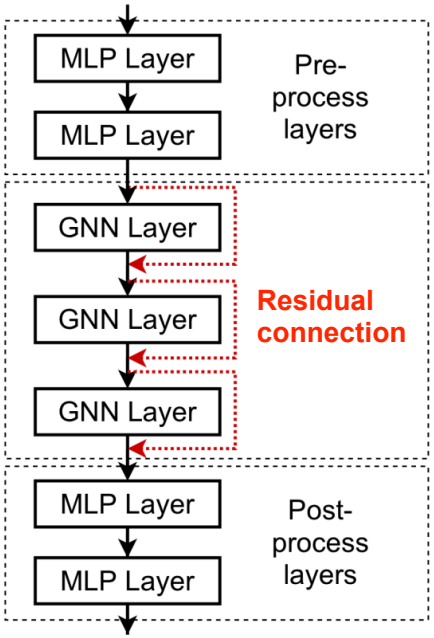
This is our $F(\mathbf{x})$



- **A GCN layer with Residual Connection**

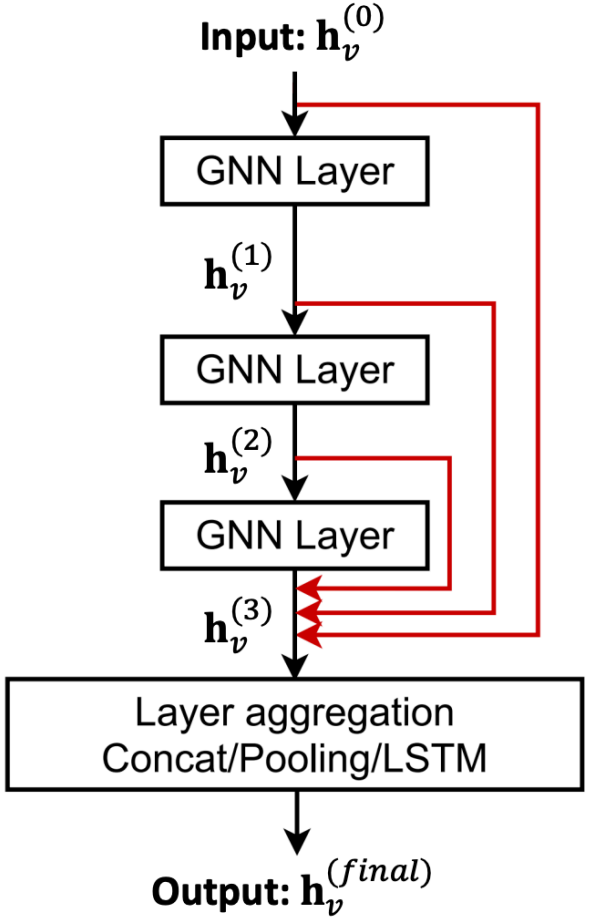
- $\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} + \mathbf{h}_v^{(l-1)}\right)$

$F(\mathbf{x})$ + $\mathbf{x}$



GCN with Jumping Knowledge Connection

- **Other options:** Directly skip to the last layer
  - The final layer directly **aggregates from the all the node embeddings** in the previous layers



How to obtain node/ edge/graph classification?

Node Classification

- **ARCHITECTURE:**
  - L message-passing layers
  - 1 readout layer (softmax function for each node):

$$y_{ni} = \frac{\exp(\mathbf{w}_i^{\mathrm{T}} \mathbf{h}_n^{(L)})}{\sum_j \exp(\mathbf{w}_j^{\mathrm{T}} \mathbf{h}_n^{(L)})}$$

where $\{w_i\}$ is a set of shared learnable weight vectors, C is the number of classes, and $i = 1..C$

- Loss function: sum of the cross-entropy loss across all nodes and all classes:

$$\mathscr{L} = - \sum_{n \in \mathscr{V}_{\text{train}}} \sum_{i=1}^{C} t_{ni} \log(y_{ni})$$

where $\{t_{ni}\}$ are target values with a one-hot encoding for each value node n

Edge classification

Typical problem: edge completion.

INTUITION: connect nodes sharing features

Compute the probability of the presence of the edge between n and m

$$p(n,m) = \sigma\left(\mathbf{h}_n^{\mathrm{T}}\mathbf{h}_m\right)$$

To train the model, we use a set of labeled edges:

$y_{n,m} = 1$ if $(n,m) \in E$ if the edge is present

$y_{n,m} = 0$ if $(n,m) \notin E$ if the edge is absent

and define the binary cross-entropy loss:

$$\mathscr{L} = -\,y_{n,m}\log p(n,m) - (1 - y_{n,m})\log(1 - p(n,m))$$

Graph classification

Task: given a training set of $\mathcal{G}_1, \ldots, \mathcal{G}_N$ labeled graphs, predict the labels of analogous graphs in test.

Architecture:

- L message-passing layers
- combine the final-layer embedding vectors in a way that does not depend on the arbitrary node ordering.
  E.g.: average or pooling of the final embeddings or:

$$\mathbf{y} = \mathbf{f}\left(\sum_{n \in \mathcal{V}} \mathbf{h}_n^{(L)}\right)$$

- MLP layers (with cross-entropy loss for classification or squared-error loss for regression)

Graph self-attention

Graph Attention network

Let recall what we have done in GCN/GraphSAGE:

$$\mathbf{z}_n^{(l)} = \text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} A_{nm} \mathbf{h}_m^{(l)}$$

$$A_{nm} \geqslant 0$$

$$\sum_{m \in \mathcal{N}(n)} A_{nm} = 1.$$

- $A_{nm} = \dfrac{1}{|\mathcal{N}(n)|}$
- defined explicitly on the base of the node degree
- all node $m \in \mathcal{N}(n)$ are equally important

Graph Attention network

Intuition: weight each neighbor according to the importance of it in influencing during the aggregation step

$$\mathbf{z}_n^{(l)} = \text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right) = \sum_{m \in \mathcal{N}(n)} A_{nm} \mathbf{h}_m^{(l)}$$

Attention of neighbor m when aggregated to node n

with $A_{nm} \geqslant 0$,

$$\sum_{m \in \mathcal{N}(n)} A_{nm} = 1.$$

Let compute A_{nm} …

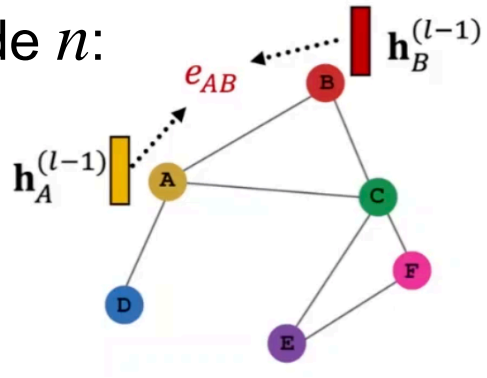
Graph Attention Network, ICLR 2018
https://arxiv.org/abs/1710.10903

Graph Attention network

- First, compute e_{nm} : the importance of m's message to node n:

$$e_{nm} = \mathbf{h}_n^T W \mathbf{h}_m$$

with $|W| = D \times D$, learnable



- Then normalize e_{nm} into the final attention weight using a softmax function

$$A_{nm} = \frac{\exp(e_{nm})}{\sum_{m' \in \mathcal{N}(n)} \exp(e_{nm'})}$$

- or combine embeddings using MLP: $A_{nm} = \frac{\exp\{\text{MLP}(\mathbf{h}_n, \mathbf{h}_m)\}}{\sum_{m' \in \mathcal{N}(n)} \exp\{\text{MLP}(\mathbf{h}_n, \mathbf{h}_{m'})\}}$

Graph Self-Attention

- A more general rule for attentive message:

$$h_i^l = W_s h_i^{l-1} + \sum_{v_j \in \mathcal{N}(v_i)} \alpha_{i,j}^{l-1} W_t h_j^{l-1}$$

Where the attentive coefficients are calculated as:

$$\alpha_{i,j} = \text{softmax}\left(\frac{(W_1 x_i)^T (W_2 x_j)}{\sqrt{d}}\right)$$

embeddings of nodes

$$\alpha_{i,j} = \text{softmax}\left(\frac{(W_1 x_i)^T (W_2 x_j + W_3 e_{i,j})}{\sqrt{d}}\right)$$

with edges

Lab Time

`GNN_graph_classification.ipynb`

`GNN_node_classification.ipynb`

References

1. **Papers;**
   - Thomas N. Kipf & Max Welling, "Semi-Supervised Classification with Graph Convolutional Networks", ICLR 2017.
   - Petar Veličković et al., "Graph Attention Networks", ICLR 2018.
   - Jure Leskovec & Rex Ying, "GraphSAGE: Inductive Representation Learning on Large Graphs", NeurIPS 2017.

2. **Book:**
   - **"Graph Representation Learning" di William L. Hamilton.**

3. **Online resources:**
   - Stanford CS224W: Network Analysis
   - DeepMind's Blog on Graph Networks: Graph Networks

4. **Framework and Toolkit:**
   - PyTorch Geometric: PyG Documentation
   - DGL (Deep Graph Library): DGL Homepage
   - TensorFlow Graph Neural Networks: TF-GNN Github